
ABOUT THE AUTHOR



Robert C. Martin (Uncle Bob) has been a programmer since 1970. He is the co-founder of cleancoders.com, which offers online video training for software developers, and is the founder of Uncle Bob Consulting LLC, which offers software consulting, training, and skill development services to major corporations worldwide. He served as the Master Craftsman at 8th Light, Inc., a Chicago-based software consulting firm. He has published dozens of articles in various trade journals and is a regular speaker at international conferences and trade shows. He served three years as the editor-in-chief of the *C++ Report* and served as the first chairman of the Agile Alliance.

Martin has authored and edited many books, including *The Clean Coder*, *Clean Code*, *UML for Java Programmers*, *Agile Software Development*, *Extreme Programming in Practice*, *More C++ Gems*, *Pattern Languages of Program Design 3*, and *Designing Object Oriented C++ Applications Using the Booch Method*.

I

INTRODUCTION

It doesn't take a huge amount of knowledge and skill to get a program working. Kids in high school do it all the time. Young men and women in college start billion-dollar businesses based on scrabbling together a few lines of PHP or Ruby. Hoards of junior programmers in cube farms around the world slog through massive requirements documents held in huge issue tracking systems to get their systems to "work" by the sheer brute force of *will*. The code they produce may not be pretty; but it works. It works because getting something to work—once—just isn't that hard.

Getting it right is another matter entirely. Getting software right is *hard*. It takes knowledge and skills that most young programmers haven't yet acquired. It requires thought and insight that most programmers don't take the time to develop. It requires a level of discipline and dedication that most programmers never dreamed they'd need. Mostly, it takes a passion for the craft and the desire to be a professional.

And when you get software right, something magical happens: You don't need hordes of programmers to keep it working. You don't need massive requirements documents and huge issue tracking systems. You don't need global cube farms and 24/7 programming.

When software is done right, it requires a fraction of the human resources to create and maintain. Changes are simple and rapid. Defects are few and far between. Effort is minimized, and functionality and flexibility are maximized.

Yes, this vision sounds a bit utopian. But I've been there; I've seen it happen. I've worked in projects where the design and architecture of the system made it easy to write and easy to maintain. I've experienced projects that required a fraction of the

anticipated human resources. I've worked on systems that had extremely low defect rates. I've seen the extraordinary effect that good software architecture can have on a system, a project, and a team. I've been to the promised land.

But don't take my word for it. Look at your own experience. Have you experienced the opposite? Have you worked on systems that are so interconnected and intricately coupled that every change, regardless of how trivial, takes weeks and involves huge risks? Have you experienced the impedance of bad code and rotten design? Has the design of the systems you've worked on had a huge negative effect on the morale of the team, the trust of the customers, and the patience of the managers? Have you seen teams, departments, and even companies that have been brought down by the rotten structure of their software? Have you been to programming hell?

I have—and to some extent, most of the rest of us have, too. It is far more common to fight your way through terrible software designs than it is to enjoy the pleasure of working with a good one.

1

WHAT IS DESIGN AND ARCHITECTURE?



There has been a lot of confusion about design and architecture over the years. What is design? What is architecture? What are the differences between the two?

One of the goals of this book is to cut through all that confusion and to define, once and for all, what design and architecture are. For starters, I'll assert that there is no difference between them. *None at all.*

The word “architecture” is often used in the context of something at a high level that is divorced from the lower-level details, whereas “design” more often seems to imply structures and decisions at a lower level. But this usage is nonsensical when you look at what a real architect does.

Consider the architect who designed my new home. Does this home have an

architecture? Of course it does. And what is that architecture? Well, it is the shape of the home, the outward appearance, the elevations, and the layout of the spaces and rooms. But as I look through the diagrams that my architect produced, I see an immense number of low-level details. I see where every outlet, light switch, and light will be placed. I see which switches control which lights. I see where the furnace is placed, and the size and placement of the water heater and the sump pump. I see detailed depictions of how the walls, roofs, and foundations will be constructed.

In short, I see all the little details that support all the high-level decisions. I also see that those low-level details and high-level decisions are part of the whole design of the house.

And so it is with software design. The low-level details and the high-level structure are all part of the same whole. They form a continuous fabric that defines the shape of the system. You can't have one without the other; indeed, no clear dividing line separates them. There is simply a continuum of decisions from the highest to the lowest levels.

THE GOAL?

And the goal of those decisions? The goal of good software design? That goal is nothing less than my utopian description:

The goal of software architecture is to minimize the human resources required to build and maintain the required system.

The measure of design quality is simply the measure of the effort required to meet the needs of the customer. If that effort is low, and stays low throughout the lifetime of the system, the design is good. If that effort grows with each new release, the design is bad. It's as simple as that.

CASE STUDY

As an example, consider the following case study. It includes real data from a real company that wishes to remain anonymous.

First, let's look at the growth of the engineering staff. I'm sure you'll agree that this trend is very encouraging. Growth like that shown in [Figure 1.1](#) must be an indication of significant success!

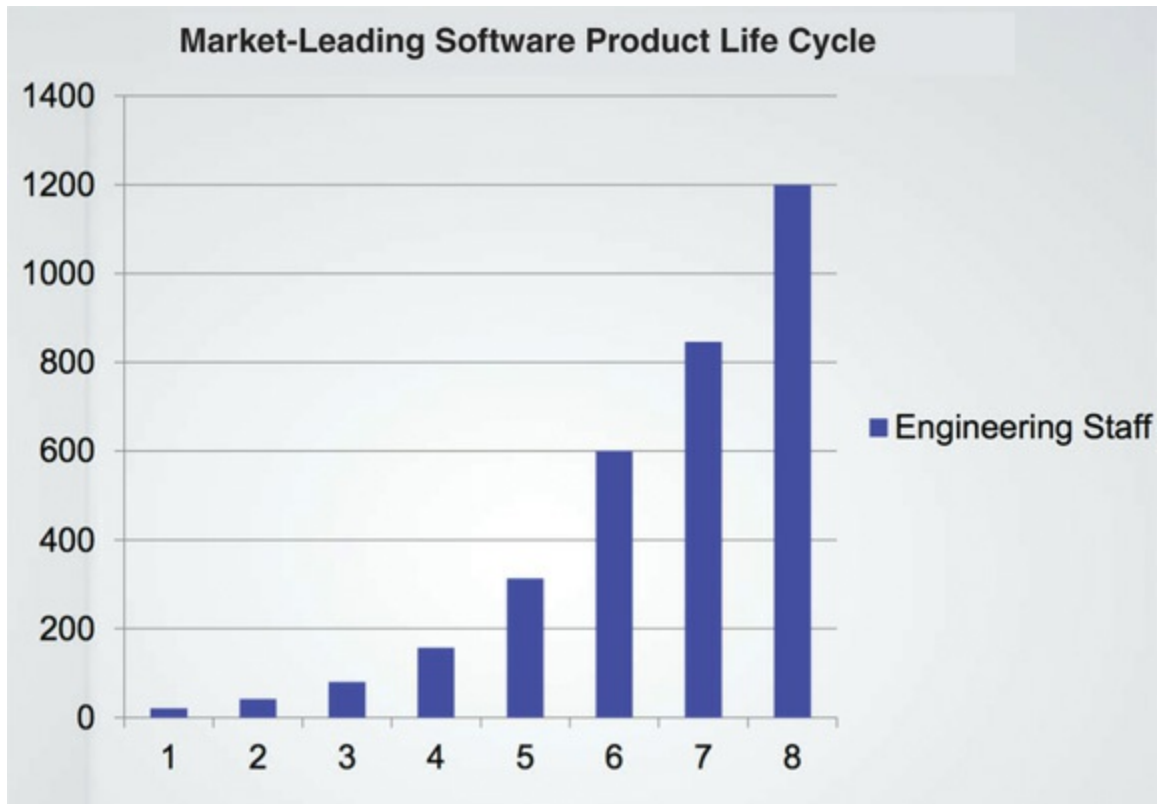


Figure 1.1 Growth of the engineering staff

Reproduced with permission from a slide presentation by Jason Gorman

Now let's look at the company's productivity over the same time period, as measured by simple lines of code ([Figure 1.2](#)).

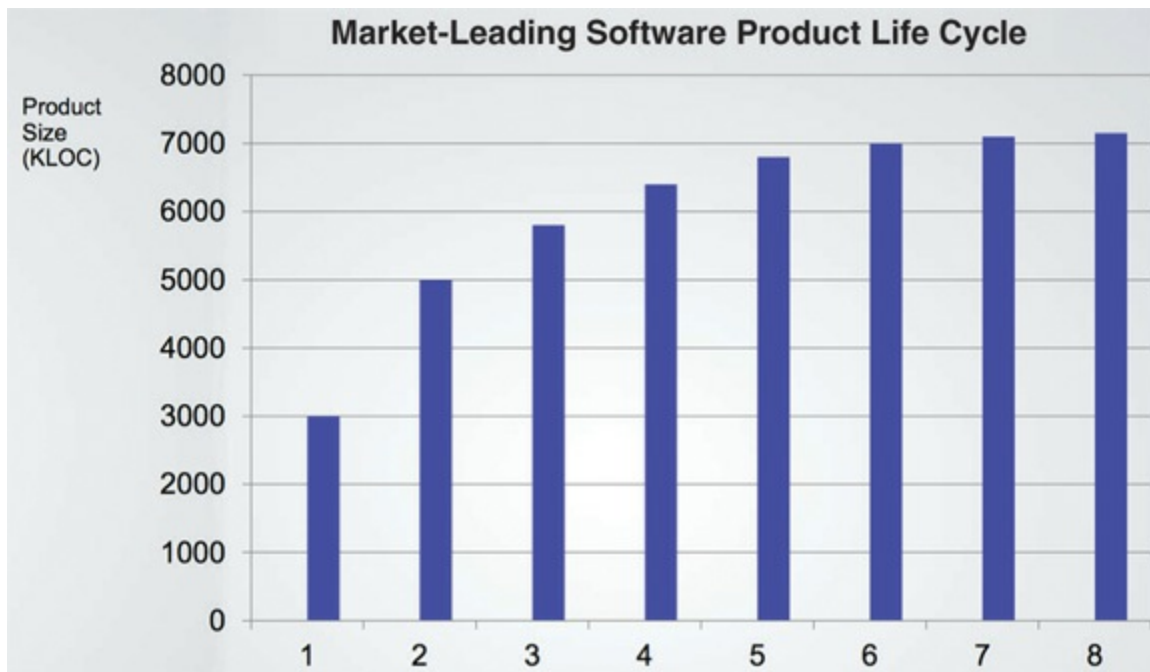


Figure 1.2 Productivity over the same period of time

Clearly something is going wrong here. Even though every release is supported by an ever-increasing number of developers, the growth of the code looks like it is approaching an asymptote.

Now here's the really scary graph: [Figure 1.3](#) shows how the cost per line of code has changed over time.

These trends aren't sustainable. It doesn't matter how profitable the company might be at the moment: Those curves will catastrophically drain the profit from the business model and drive the company into a stall, if not into a downright collapse.

What caused this remarkable change in productivity? Why was the code 40 times more expensive to produce in release 8 as opposed to release 1?

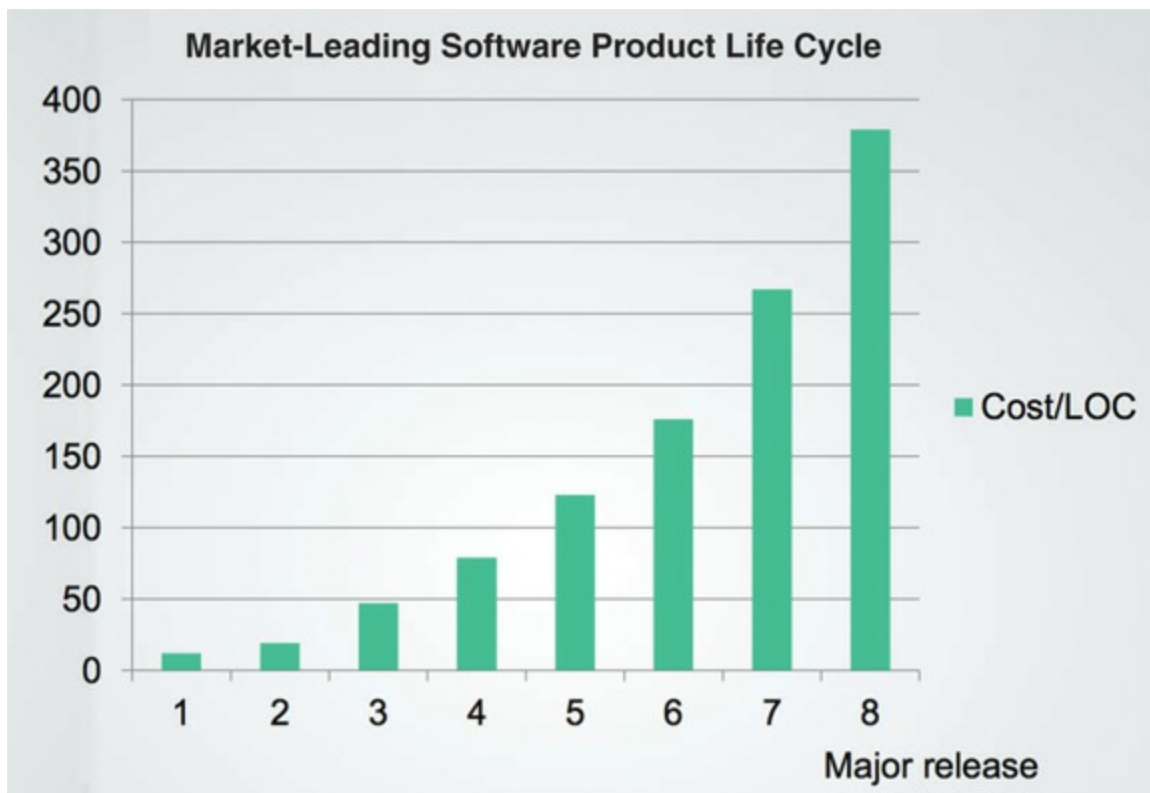


Figure 1.3 Cost per line of code over time

THE SIGNATURE OF A MESS

What you are looking at is the signature of a mess. When systems are thrown together in a hurry, when the sheer number of programmers is the sole driver of

output, and when little or no thought is given to the cleanliness of the code or the structure of the design, then you can bank on riding this curve to its ugly end.

[Figure 1.4](#) shows what this curve looks like to the developers. They started out at nearly 100% productivity, but with each release their productivity declined. By the fourth release, it was clear that their productivity was going to bottom out in an asymptotic approach to zero.

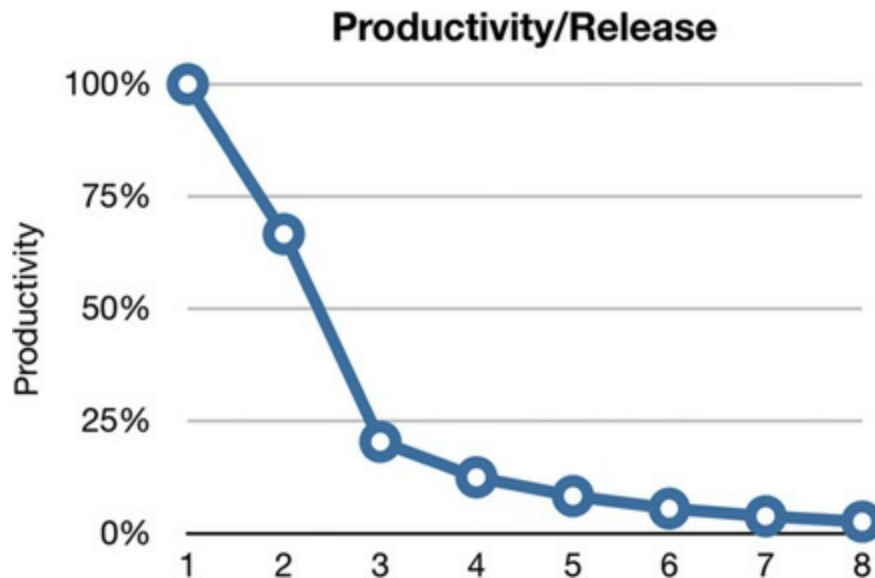


Figure 1.4 Productivity by release

From the developers' point of view, this is tremendously frustrating, because everyone is working *hard*. Nobody has decreased their effort.

And yet, despite all their heroics, overtime, and dedication, they simply aren't getting much of anything done anymore. All their effort has been diverted away from features and is now consumed with managing the mess. Their job, such as it is, has changed into moving the mess from one place to the next, and the next, and the next, so that they can add one more meager little feature.

THE EXECUTIVE VIEW

If you think *that's* bad, imagine what this picture looks like to the executives! Consider [Figure 1.5](#), which depicts monthly development payroll for the same period.